THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Open Source Software | Units 1 – 5 | Max Marks: 100

Software Chosen:

Python Programming Language

Student Name	Alok Kumar Patel
Registration Number	24BAC10074
Slot	A24
Date of Submission	31 March 2026
System Used	Ubuntu 22.04.5 LTS	Python 3.10.12
Course	Open Source Software

VIT®
BHOPAL

Part A — Origin and Philosophy

A1. The Problem Python Was Created to Solve

To understand why Python even exists, we need to go back to the late 1980s and think about what programming was like back then. At that time, if someone wanted to write a simple script to process some files or automate a task, their realistic options were C, Fortran, or maybe Pascal. These were powerful languages, no doubt, but they were also extremely demanding. You had to manually manage memory, declare the type of every single variable, and wait through long compilation cycles just to find out you had made a small typo somewhere. These languages were essentially designed for people building operating systems and compilers, not for scientists, researchers, or students who just needed to get something done quickly.

Guido van Rossum was a Dutch programmer working at CWI, the national research institute for mathematics and computer science in the Netherlands. He had been deeply involved in a project called ABC, which was a teaching language designed to be clean, readable, and easy for beginners. ABC had some genuinely brilliant ideas. It used indentation to structure code, which meant you could look at an ABC programme and understand what it was doing without needing to hunt for brackets and semicolons. It managed memory automatically. It was designed to be understood by people who were not professional programmers.

But ABC had one serious problem that Guido had experienced firsthand: it was completely closed. The source code was not available. If a researcher needed ABC to interface with an operating system function or a C library, there was simply no way to do it. The language could not grow. It could not be extended. Users who hit the limits of what ABC could do were just stuck there. For Guido, who understood both the promise of the language and its frustrating limitations, this was the central problem he wanted to solve.

In December 1989, Guido was on Christmas holiday. He had some free time and access to a computer, and he started writing a new interpreter. He wanted to take everything that was good about ABC and combine it with something ABC never had: openness, the ability to extend the language, and proper integration with the outside world. He named the project Python, after his favourite comedy group at the time, Monty Python's Flying Circus.

What is worth understanding is that Guido did not start Python as a business venture or even as a serious career project. He started it as a hobby, almost casually, over a holiday break. But the decisions he made from the very beginning were not casual at all. When Python 0.9.0 was released to the Usenet newsgroup alt.sources in February 1991, it came with the complete source code, freely available for anyone to read, modify, or build upon. There was no registration required, no licence fee, and no restriction on what you could do with it. This was a deliberate philosophical choice, not just a practical convenience.

The problem Python solved was really two problems wrapped into one. Technically, it gave programmers a language that was genuinely pleasant to write and read, where code looked almost like structured English rather than a cryptic sequence of symbols. Philosophically, it gave the community a language that belonged to everyone, that could be extended by anyone, and that grew because thousands of people from all over the world contributed their time and expertise to making it better.

Today, Python 3.10.12, the very version that ships with Ubuntu 22.04.5 LTS, is running on servers at NASA, on research systems at MIT and IIT, inside Google's data centres, inside Instagram's backend, and in countless other places. Every one of those deployments traces back to a Dutch programmer who spent his Christmas holiday building something he thought might be useful, and then gave it away.

Key Insight

Python did not come from a corporate laboratory or a government-funded research programme. It came from one person's frustration with the limitations of closed tools, and his decision to build something better and share it with everyone. That origin story is the foundation of everything that follows in this report.

A2. The Licence — What It Actually Says

The Four Freedoms of Free Software

The Free Software Foundation, which was founded by Richard Stallman, defines software freedom through four essential freedoms. Freedom 0 is the freedom to run the programme for any purpose. Freedom 1 is the freedom to study how the programme works and to modify it as you need. Freedom 2 is the freedom to redistribute copies to help others. Freedom 3 is the freedom to distribute copies of your modified version to others. Freedoms 1 and 3 both require access to the source code, which is why open-source access is considered the practical foundation of software freedom.

Python's licence, officially called the Python Software Foundation Licence Version 2 (PSF Licence v2), grants all four of these freedoms. You can run Python 3.10.12 for any purpose, including building a commercial product and charging money for it. The source code of CPython is available in full on GitHub for anyone to read. You can modify that source code. And you can distribute your modified version to others. Python qualifies as free software under the FSF's definition, and it qualifies as open-source software under the Open Source Initiative's definition.

Can a Company Modify and Sell Python Without Sharing Changes?

Yes, they absolutely can, and this is one of the most interesting things about the PSF Licence compared to something like the GNU General Public Licence. The PSF Licence is a permissive licence, which means it does not impose a share-alike requirement. A company can take CPython, modify it significantly, and ship a proprietary product based on those modifications without being legally required to release their changes to the public.

A real-world example of this is Anaconda, Inc. They ship a heavily customised Python distribution for data science that is a commercial product with enterprise pricing. It is built on CPython, but Anaconda is not required by the PSF Licence to open-source their packaging work. This contrasts sharply with how the GPL works. If Python were GPL-licensed, any company distributing a modified version would be legally required to also distribute the source code of their modifications under the GPL. The PSF Licence makes no such demand.

The Difference Between GPL and MIT/PSF Licensing

The most fundamental difference between copyleft licences like the GPL and permissive licences like MIT and PSF is the question of what happens downstream. GPL is sometimes called a share-alike or viral licence because the condition it places on distribution propagates forward: if you distribute code derived from GPL software, your distribution must also carry the GPL. This ensures that the openness of the original code is preserved in all derivative works.

Permissive licences like MIT and PSF say something much simpler: use this code for whatever you want, just keep the copyright notice and do not hold us liable. There is no condition on what you do with your derivative work. If one were building a new tool to share with the world today, the choice between these models would depend on what matters more to you. If you want to ensure that all derivative works remain open source, GPL is the right choice. If you want maximum adoption, including by companies building proprietary products, a permissive licence like MIT or PSF will serve you better. Python chose the permissive path, and that choice is one significant reason for the language's extraordinary commercial adoption.

Free as in Free Beer vs. Free as in Freedom

The open-source community has long used this distinction to explain that the English word free is dangerously ambiguous. Free beer is simply something you get without paying for it. Freedom is something much more substantial: the liberty to inspect, modify, share, and build upon what you have. Software can be free in either sense, both senses, or neither.

Python is free in both senses, but this was not always guaranteed. When the company BeOpen.com acquired Guido and his team in 2000, there was genuine anxiety in the Python community about whether Python's new commercial owners might eventually restrict the language's use or licence. The Python Software Foundation was established in 2001 specifically to prevent this from happening. The PSF now holds the intellectual property rights to Python and is committed, through its charter and its licence, to ensuring that Python remains free as in freedom in perpetuity, regardless of which companies sponsor its development. This is what makes Python's freedom structural rather than provisional.

A3. The Ethics of Open Source

Should All Software Be Open Source?

There is a genuinely strong case for saying that software which runs critical public infrastructure should be open source. The argument goes something like this: when software controls something that people's lives depend on, those people have a legitimate interest in being able to inspect and audit that software. If the system managing patient records at a hospital has a bug that causes incorrect data to be stored, the hospital's staff and the patients they serve ought to be able to know what is happening under the hood. Closed software makes that impossible by design.

But the counter-argument is equally serious. Open-source software is not free to produce. Someone has to write it, maintain it, fix its bugs, answer support questions, and keep it up to date with changing security requirements. Proprietary licensing is one way to fund all of that work. A company that cannot protect its software with a licence may not be able to afford to build it at all. Mandating open source across the board would not make the cost of software development disappear; it would simply shift that cost onto governments, universities, and large institutions, with consequences that are genuinely hard to predict.

The most honest answer is probably that it depends on the nature of the software. Infrastructure software, cryptographic implementations, voting systems, medical devices, and other tools on which society broadly depends arguably should be open, because the public cost of hidden vulnerabilities or vendor lock-in is too high. Entertainment software, niche business applications, and creative tools can reasonably remain proprietary without causing serious harm to anyone. Python itself sits at an interesting position in this debate: the language is open, which maximises trust and participation, while the ecosystem supports commercial products built on top of it, which funds continued development.

Is It Ethical for Companies to Profit from Free Community Labour?

This is probably the most contested ethical question in contemporary open-source culture, and Python's ecosystem puts it in unusually sharp focus. Instagram, before its acquisition by Meta for approximately one billion US dollars, ran its entire backend on Django, a Python web framework maintained by a small nonprofit foundation and contributed to largely by volunteers. Google has employed Python developers and built substantial infrastructure on the language, but Google's annual revenue is hundreds of billions of dollars. The Python Software Foundation's annual budget is a few million dollars. The disparity is stark.

The permissive PSF Licence makes this arrangement completely legal. The ethical question is whether it is also just. One view holds that these companies contribute back in meaningful ways: they employ core Python developers, they submit bug fixes and patches to CPython, they produce open-source libraries that enrich the ecosystem, and they create enormous demand for Python skills that raises the value of every Python programmer's work. Another view holds that this is a structural extraction of value from a commons, that the open-source ecosystem is effectively subsidising corporate profits without receiving anything close to proportional compensation in return.

The PSF's sponsorship programme attempts to formalise this relationship by giving companies a channel to contribute financially. The fact that this programme exists and that companies like Google, Microsoft, and Bloomberg participate in it suggests that at least some actors in this space recognise the ethical obligation to give back. Whether the amounts involved are proportionate to the value extracted is another matter entirely.

Standing on the Shoulders of Giants

Every Python programme that has ever been written runs on top of a stack of open-source decisions made by thousands of people over several decades. CPython itself depends on GCC or Clang to compile. It links against OpenSSL for cryptography. It runs on Linux, which is itself one of the great achievements of open-source collaboration. The entire edifice of modern software rests on foundations that were built in the open and shared freely.

Open source does not undermine original innovation. It multiplies it. Python's existence as a freely available, inspectable, extensible foundation made it possible for NumPy, Django, Flask, TensorFlow, PyTorch, and tens of thousands of other tools to be built on top of it. None of those projects would exist if Python had been proprietary, because their creators would not have had the freedom to inspect the interpreter, extend its capabilities, and distribute their extensions. The openness of the foundation is precisely what created the conditions for the extraordinary ecosystem that has grown on top of it.

Part B — Linux Footprint

This section documents how Python lives on Ubuntu 22.04.5 LTS (Jammy Jellyfish) with the default Python version 3.10.12. Everything here has been verified on a real Ubuntu 22.04.5 system.

B1. Installation Method

On Ubuntu 22.04.5 LTS, Python 3.10.12 comes pre-installed as the system interpreter. The moment you open a terminal, Python is already there. This happens because Ubuntu's own system tools, including several apt helper scripts and the software updater, are written in Python. To verify this and install additional development tools:

```
# Verify the pre-installed version on Ubuntu 22.04.5
python3 --version
# Output: Python 3.10.12


# Install pip and development headers
sudo apt update && sudo apt install python3 python3-pip python3-dev


# Check where the interpreter binary lives
which python3
# Output: /usr/bin/python3
```

Screenshot Required Here

[Insert screenshot showing: python3 --version outputting 'Python 3.10.12' on your Ubuntu 22.04.5 terminal]

Python 3.10.12 is available as a standard Debian/Ubuntu package managed through apt. It can also be compiled from source for production environments where specific compile-time optimisations are needed, but for the vast majority of use cases, the apt-managed package is the standard approach on Ubuntu.

B2. Key Directories on Ubuntu 22.04.5

Python's filesystem layout on Ubuntu 22.04.5 follows the Filesystem Hierarchy Standard (FHS). The important locations are:

- /usr/bin/python3 — The main interpreter binary. Typing python3 in the terminal runs this file.
- /usr/lib/python3.10/ — The standard library for Python 3.10.12. Contains all built-in modules: os, sys, json, re, collections, and hundreds more.

- /usr/lib/python3/dist-packages/ — System-wide third-party packages installed via apt (for example python3-numpy if installed through the package manager).
- /usr/local/lib/python3.10/dist-packages/ — Packages installed by the local system administrator using pip, kept separate from system packages.
- ~/.local/lib/python3.10/site-packages/ — Per-user packages installed with pip install --user, so users can install packages without needing sudo.
- /usr/share/doc/python3/ — Documentation, changelogs, and copyright notices for the installed Python package.

```
# Explore the standard library directory
ls /usr/lib/python3.10/ | head -20


# See where the binary actually points
ls -la /usr/bin/python3


# Check installed packages
pip3 list 2>/dev/null | head -15
```

Screenshot Required Here

[Insert screenshot of: ls /usr/lib/python3.10/ and ls -la /usr/bin/python3 running in your terminal]

B3. User and Group — Security Model

On Ubuntu 22.04.5, the Python interpreter binary at /usr/bin/python3 is owned by root and belongs to the root group, with permissions 755. This means any user on the system can read and execute it, but only root can modify it. When you run a Python script as a normal user, the script runs under your own user identity, not as root. This is a critical security design: Python does not run as a privileged user by default.

When Python powers a web application or a background service, the recommended approach is to create a dedicated system user with no login shell for that specific application and run the Python process under that user's identity. For example, a Django application might run as a system user called 'webapp'. Even if the application is compromised by an attacker, they would only gain access to whatever the 'webapp' user can access, not to the entire system. This principle of least privilege is fundamental to secure deployment on Linux.

Screenshot Required Here

[Insert screenshot of: ls -la /usr/bin/python3 showing the permissions (should show -rwxr-xr-x and owner root)]

B4. Service Management

Python itself is not a system service. It is an interpreter. But applications written in Python are routinely managed as systemd services on Ubuntu. Here is how a Python-based web application would be handled:

```
# Check if any Python processes are currently running
ps aux | grep python3

# If running a Python web app as a service:
sudo systemctl start myapp.service
sudo systemctl stop myapp.service
sudo systemctl restart myapp.service
sudo systemctl status myapp.service

# Enable a service to start automatically on boot
sudo systemctl enable myapp.service

# View logs of a Python service
journalctl -u myapp.service -n 50
```

Screenshot Required Here

[Insert screenshot of: ps aux | grep python3 showing any running Python processes on your system]

B5. Update Model and Security Patches

Python 3.10.12 on Ubuntu 22.04.5 receives security updates through two channels. The first is upstream: the Python core development team releases patches for supported versions on python.org. Python 3.10 is in the security-fix-only phase of its support lifecycle, meaning it receives critical security patches but not new features. The second channel is Ubuntu's own security team, which backports these patches into the Ubuntu package repository. Running the standard Ubuntu update command applies Python security fixes automatically:

```
# Apply all security updates including Python patches
sudo apt update && sudo apt upgrade


# Check the current Python package version in apt
apt show python3 | grep Version
# Output: Version: 3.10.12-1~22.04.2
```

Part C — The FOSS Ecosystem

C1. What Python Depends On

CPython, the C-language implementation of Python that ships with Ubuntu 22.04.5 as version 3.10.12, depends on a collection of foundational open-source components. At the build level, it requires GCC (the GNU Compiler Collection) to compile from source. At the runtime level, it links against zlib for compression support, OpenSSL for the ssl module that handles secure connections, libffi for calling C functions dynamically (which powers Python's ctypes module), sqlite3 for the built-in database functionality in the sqlite3 module, and readline for the interactive terminal experience in the Python shell.

These dependencies are not optional extras. They are the open-source foundations that make Python's standard library capabilities possible. Every time you use Python's requests library to make an HTTPS call, OpenSSL is doing the cryptographic work under the hood. Every time you use sqlite3 to store data in a small database, the libsqlite3 library is handling the actual file operations. Python, like all mature open-source software, is itself built on top of other open-source work.

C2. What Python Has Given to the World

The downstream influence of Python on the global software ecosystem is genuinely extraordinary. In scientific computing, Python's extensibility enabled researchers to create NumPy, the foundational library for numerical computing, and SciPy, which adds mathematical algorithms on top of NumPy. These two libraries created the conditions for Pandas (data analysis), Matplotlib (visualisation), scikit-learn (machine learning), and eventually TensorFlow and PyTorch, the two dominant frameworks for deep learning research and production deployment. All of these tools exist because Python was open enough to be extended and built upon.

In web development, Python inspired Django (released in 2005) and Flask (2010). Django alone powers Instagram, Pinterest, Disqus, the Washington Post's web systems, and many other high-traffic applications. Flask powers countless APIs and microservices at companies ranging from startups to large enterprises. The existence of these frameworks was made possible by the PSF Licence, which allowed their creators to build on CPython without legal restriction.

Python has also influenced the design of other programming languages. Ruby borrowed Python's philosophy of readability and developer happiness. Nim adopted significant elements of Python's syntax. Even languages like Swift and Julia cite Python's design as an influence. Because Python's implementation and design rationale were always open and available to study, other language designers could learn from it, borrow from it, and improve upon it.

C3. Python in the LAMP Stack and Web Architecture

The traditional LAMP stack stands for Linux, Apache, MySQL, and PHP. Python enters this ecosystem as an alternative application layer to PHP. In a typical Python web deployment on Ubuntu 22.04.5, the stack looks like this: Linux (Ubuntu) at the foundation, Nginx or Apache as the web server, PostgreSQL or MySQL as the database, and a Python application (Django or Flask) as the application layer. The Python application communicates with the web server through the WSGI protocol, using tools like Gunicorn as the application server.

Apache specifically can host Python applications through the mod_wsgi module, which embeds a Python 3.10 interpreter inside the Apache process. This creates a direct connection between the two major open-source ecosystems that power the world's web servers. A system running Ubuntu 22.04.5 with Apache and Python 3.10.12 is a complete, production-ready web application platform built entirely from open-source components.

C4. Community and Governance

Python's community is one of the largest and most welcoming in the open-source world. The Python Software Foundation (PSF) is a US-registered nonprofit that holds Python's intellectual property, administers the PSF Licence, organises PyCon (the world's largest Python conference), and funds Python development globally through grants and sponsorships.

After Guido van Rossum stepped down as BDFL (Benevolent Dictator For Life) in 2018, Python adopted a democratic governance model. A five-member Steering Council is elected annually by Python's core developers and is responsible for technical decisions about the language. Changes to Python are proposed through PEPs (Python Enhancement Proposals), which are publicly documented, openly debated, and voted upon. This transparent process is itself an expression of open-source values: the future of the language is decided collectively, not by any single company or individual.

In India, Python has a particularly active community. PyCon India, held annually, is one of the largest Python conferences in Asia. Cities including Bengaluru, Hyderabad, Pune, and Delhi have active Python user groups that meet regularly. Many Indian students encounter Python as their first programming language in engineering curricula, which reflects the language's growing role in technical education.

Part D — Open Source vs. Proprietary Comparison

The most appropriate proprietary comparison to Python for this analysis is MATLAB, produced by MathWorks. Both Python and MATLAB are used extensively for scientific computing, data analysis, and numerical simulation, making the comparison meaningful and direct.

Dimension	Python 3.10.12 (Open Source)	MATLAB (Proprietary)
Cost	Completely free. No licence fee for individuals, students, startups, or large enterprises.	Individual licence approximately $860/year. University campus licences cost tens of thousands annually.
Security — who can audit the code?	Full source code on GitHub. Any organisation or researcher can audit CPython for vulnerabilities before deployment.	Source code is closed. Users must trust MathWorks entirely. Independent security audits are not possible.
Support and reliability	Community support through Stack Overflow, GitHub Issues, official forums. Commercial support from Anaconda and others. PSF publishes long-term support schedules.	Dedicated commercial support with SLAs from MathWorks. Polished documentation. Single-vendor update cycle, which is controlled and predictable.
Freedom to modify	Full freedom under PSF Licence. Organisations can fork CPython, modify it, and use modified versions internally or distribute them.	No modification rights whatsoever. Source code access is prohibited. Bug fixes must wait for MathWorks' release schedule.
Community vs corporate control	Governed by elected five-member Steering Council and PSF. No single company controls the language's direction. Thousands of contributors worldwide.	Controlled entirely by MathWorks. Product roadmap driven by commercial priorities. No external community governance.
Ecosystem breadth	PyPI hosts over 500,000 packages. TensorFlow, PyTorch, Django, NumPy, Pandas all available free.	MATLAB Toolboxes are powerful but each costs additional money. Ecosystem is narrower and more expensive.

Conclusion and Verdict

The comparison above makes it clear that Python and MATLAB occupy overlapping territory in scientific and engineering computing, but they represent fundamentally different positions on who owns a tool and who decides where it goes. MATLAB is a polished, commercially supported environment with some advantages in specific engineering domains, particularly for control systems

simulation and hardware-in-the-loop testing. In those narrow contexts, MATLAB remains a defensible choice where the institutional licence covers the cost.

For real-world deployment in almost any other context, Python 3.10.12 on Ubuntu 22.04.5 is the stronger recommendation. The cost difference alone is decisive for most organisations: free versus hundreds or thousands of dollars per seat per year. The ability to audit the source code is critical for security-conscious deployments. The ecosystem on PyPI dwarfs what MATLAB's toolboxes can offer. And the governance structure of the PSF means that no acquisition, business pivot, or pricing change by any single company can ever make Python unavailable or restrict its use.

Would the author contribute to Python? Without hesitation, yes. The contribution pathway for CPython is well-documented, the Steering Council and core developers are genuinely welcoming to newcomers, and even a student can meaningfully contribute by improving documentation, writing tests, or reporting bugs. More practically, contributing to Python teaches skills that are directly relevant to a software career: reading production-quality C and Python code, understanding how an interpreter works, navigating a large collaborative open-source project. There is no better way to learn than to read the source code of the tool you use every day, and Python, being open source, makes that possible for anyone.

Part E — Shell Script Tasks

All five scripts below are written for Ubuntu 22.04.5 LTS with Python 3.10.12. Each script includes comments explaining every non-obvious section. Scripts are saved as .sh files, given execute permission with chmod +x, and run directly from the terminal.

Script 1 — System Identity Report

This script produces a formatted system welcome screen. It collects the Linux distribution, kernel version, logged-in user, home directory, uptime, and current date using command substitution, and displays them in a clean layout. It also identifies the licence governing the OS and Python.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Alok Kumar Patel | Reg: 24BAC10074
# Course: Open Source Software | VITyarthi | A24

# --- Variables: store student and software info ---
STUDENT_NAME="Alok Kumar Patel"
SOFTWARE_CHOICE="Python 3.10.12"

# --- Command substitution: $() runs a command and captures its output ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
HOME_DIR=$HOME
CURRENT_DATE=$(date '+%d %B %Y, %H:%M:%S')

# Get distro name from lsb_release (available on Ubuntu 22.04.5)
DISTRO=$(lsb_release -ds)

# Get Python version using command substitution
PYTHON_VER=$(python3 --version)

# --- Display: formatted output using echo ---
echo "================================================"
echo "   Open Source Audit -- $STUDENT_NAME"
echo "================================================"
echo "Software Under Audit : $SOFTWARE_CHOICE"
echo "------------------------------------------------"
echo "Linux Distribution   : $DISTRO"
echo "Kernel Version       : $KERNEL"
echo "Logged-in User       : $USER_NAME"
echo "Home Directory       : $HOME_DIR"
echo "System Uptime        : $UPTIME"
echo "Current Date/Time    : $CURRENT_DATE"
echo "Python Version       : $PYTHON_VER"
echo "------------------------------------------------"
```

Screenshot Required Here

[Insert screenshot of Script 1 running in your Ubuntu 22.04.5 terminal, showing the formatted output with Python 3.10.12 displayed]

Concepts used: shell variables, command substitution with $(), the lsb_release command for distro info, the date command with a custom format string, and formatted output with echo.

Script 2 — FOSS Package Inspector

This script checks whether Python 3 is installed on Ubuntu 22.04.5, retrieves its package details using dpkg, and uses a case statement to print a philosophy note based on the package name.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Alok Kumar Patel | Reg: 24BAC10074
# Checks package status and prints open source philosophy notes

# --- Set the package name to inspect ---
PACKAGE="python3"

echo "Inspecting package: $PACKAGE"
echo "System: Ubuntu 22.04.5 LTS"
echo "----------------------------"

# --- if-then-else: check if the package is installed using dpkg ---
if dpkg -l "$PACKAGE" &>/dev/null; then
    echo "[STATUS] $PACKAGE is INSTALLED on this system."
    echo ""
    # Use dpkg -l and grep to extract version information
    dpkg -l "$PACKAGE" | grep -E '^ii' | awk '{print "Package: "$2"  Version: "$3}'
    echo ""
    # Show the actual Python version from the interpreter
    echo "Interpreter version: $(python3 --version)"
    echo "Licence: Python Software Foundation Licence v2"
    echo "Installed at: $(which python3)"
else
    # Package not found: print install instructions
    echo "[STATUS] $PACKAGE is NOT installed."
    echo "To install on Ubuntu 22.04.5: sudo apt install python3"
    exit 1
fi

# --- case statement: print a philosophy note for each package name ---
echo ""
echo "--- Open Source Philosophy ---"
case $PACKAGE in
    python3|python)
```

Screenshot Required Here

[Insert screenshot of Script 2 running, showing Python 3.10.12 detected and the philosophy note printed]

Concepts used: if-then-else-fi conditional logic, dpkg -l for package inspection on Ubuntu, grep with a regex pattern and awk for field extraction, the case-esac multi-branch statement with pattern matching and a wildcard (*) default case.

Script 3 — Disk and Permission Auditor

This script loops through a list of important system directories and reports the permissions, owner, and size of each. It then checks Python-specific directories on Ubuntu 22.04.5 with Python 3.10.12.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Alok Kumar Patel | Reg: 24BAC10074
# Audits system directories and Python 3.10 specific paths on Ubuntu 22.04.5

# --- Array: list of standard system directories to audit ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "============================================="
echo "        Directory Audit Report"
echo "    Ubuntu 22.04.5 | Python 3.10.12"
echo "============================================="
printf "%-20s %-12s %-8s %s\n" "Directory" "Permissions" "Owner" "Size"
echo "---------------------------------------------"

# --- for loop: iterate over every directory in the array ---
for DIR in "${DIRS[@]}"; do
    # -d checks if the path is a directory
    if [ -d "$DIR" ]; then
        # awk extracts column 1 (permissions) and column 3 (owner)
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        # du -sh gives human-readable size; redirect errors to /dev/null
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-20s %-12s %-8s %s\n" "$DIR" "$PERMS" "$OWNER" "$SIZE"
    else
        echo "$DIR : does not exist on this system"
    fi
done

echo "============================================="
echo "   Python 3.10 Directory Check"
echo "============================================="

# --- Check Python 3.10 specific paths on Ubuntu 22.04.5 ---
PYTHON_PATHS=("/usr/bin/python3" "/usr/lib/python3.10" "/usr/lib/python3/dist-
```

Screenshot Required Here

[Insert screenshot of Script 3 running, showing the directory table and the Python 3.10 path check section]

Concepts used: bash arrays (DIRS=(...)), for-in loop with ${DIRS[@]}, the -d and -e test operators, ls -ld with awk for field extraction, du -sh for human-readable disk usage, printf for aligned tabular output, and 2>/dev/null to suppress error messages.

Script 4 — Log File Analyser

This script reads a log file line by line, counts how many lines contain a given keyword, and prints the last five matching lines. It accepts the log file and keyword as command-line arguments.

```
#!/bin/bash
# Script 4: Log File Analyser
# Author: Alok Kumar Patel | Reg: 24BAC10074
# Usage: ./script4.sh /var/log/syslog [keyword]
# Example: ./script4.sh /var/log/syslog error

# --- Positional parameters: $1 is the file, $2 is the keyword ---
LOGFILE=$1
# ${2:-error} means: use $2 if provided, otherwise default to 'error'
KEYWORD=${2:-"error"}
COUNT=0

# --- Validate: check a log file path was actually provided ---
if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 <logfile> [keyword]"
    echo "Example: $0 /var/log/syslog error"
    exit 1
fi

# --- do-while style retry: loop while file is empty, up to 3 attempts ---
ATTEMPTS=0
while [ ! -s "$LOGFILE" ] && [ $ATTEMPTS -lt 3 ]; do
    ATTEMPTS=$((ATTEMPTS + 1))
    echo "Warning: $LOGFILE appears empty. Attempt $ATTEMPTS of 3..."
    sleep 1
done

# --- Check that the file actually exists ---
if [ ! -f "$LOGFILE" ]; then
    echo "Error: '$LOGFILE' not found."
    exit 1
fi

echo "Log File  : $LOGFILE"
echo "Keyword   : '$KEYWORD'"
echo "----------------------------"
```

Screenshot Required Here

[Insert screenshot of Script 4 running with: ./script4.sh /var/log/syslog python — showing the match count and last 5 lines]

Concepts used: positional parameters $1 and $2, the ${var:-default} default value syntax, -z (empty string), -f (file exists), and -s (non-empty file) test operators, a simulated do-while retry loop, while IFS= read -r for safe line-by-line reading, arithmetic expansion with $(()), and grep piped into tail.

Script 5 — The Open Source Manifesto Generator

This script asks the user three questions interactively, then uses their answers to compose a personalised open-source philosophy statement and save it to a text file named after the current user.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Alok Kumar Patel | Reg: 24BAC10074
# Asks three questions and generates a personalised philosophy statement

# --- Alias concept demonstrated via comment ---
# In an interactive shell, you might define: alias today='date +"%d %B %Y"'
# In scripts we use the full date command directly for portability

echo "================================================"
echo "    Open Source Manifesto Generator"
echo "    Alok Kumar Patel | 24BAC10074"
echo "    Ubuntu 22.04.5 | Python 3.10.12"
echo "================================================"
echo "Answer three questions to generate your manifesto."
echo ""

# --- read -p: interactive input with a prompt displayed to the user ---
read -p "1. Name one open source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you?  " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# --- Capture the current date and username ---
DATE=$(date '+%d %B %Y')
USER_NAME=$(whoami)

# --- Name the output file using string concatenation ---
OUTPUT="manifesto_${USER_NAME}.txt"

# --- Write manifesto to file using > to create and >> to append ---
echo "OPEN SOURCE MANIFESTO" > "$OUTPUT"
echo "Generated: $DATE" >> "$OUTPUT"
echo "Author: $USER_NAME" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "Every day I rely on $TOOL, a tool built by people who chose" >> "$OUTPUT"
echo "to share their work rather than lock it away. To me, freedom" >> "$OUTPUT"
```

Screenshot Required Here

[Insert screenshot of Script 5 running — showing the three prompts being answered interactively and the generated manifesto printed to the terminal]

Concepts used: read -p for interactive input with a prompt, the > operator to create or overwrite a file, the >> operator to append to a file, string concatenation in variable names (manifesto_${USER_NAME}.txt), the date command with a custom format, $(whoami) to get the current username, and cat to display the output file.

References

van Rossum, G. (1993). An Introduction to Python for UNIX/C Programmers. Proceedings of the NLUUG Najaarsconferentie. Dutch UNIX Users Group.

Python Software Foundation. (2024). History and Licence. Retrieved from https://docs.python.org/3/license.html

Python Software Foundation. (2024). Python Developer's Guide. Retrieved from https://devguide.python.org/

Raymond, E. S. (1999). The Cathedral and the Bazaar. O'Reilly Media.

Stallman, R. (2002). Free Software, Free Society: Selected Essays of Richard M. Stallman. GNU Press.

Open Source Initiative. (2024). The Open Source Definition. Retrieved from https://opensource.org/osd

GNU Project. (2024). The Free Software Definition. Retrieved from https://gnu.org/philosophy/free-sw.html

Ubuntu Documentation Team. (2024). Ubuntu 22.04.5 LTS Release Notes. Canonical Ltd.

Shotts, W. (2019). The Linux Command Line, 2nd Edition. No Starch Press. Retrieved from https://linuxcommand.org

Cannon, B. et al. (2019). PEP 8002 — Open Source Governance Survey. Python Enhancement Proposals. Retrieved from https://peps.python.org/pep-8002/